SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Deep Learning COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BL3)

QUESTIONS: 1

Duration: 1 Hour
Total Marks:50

Annexure A

Error Analysis Task

1.A neural network predicts student scores with an MSE of 0.85 during training and 5.20 during testing. Analyse the possible reasons for this difference and suggest suitable corrective measures.

Analysis of Training and Testing MSE in a Neural Network

A neural network is used to predict student scores. During training, the model achieves a **Mean Squared Error (MSE) of 0.85**, while during testing the MSE increases to **5.20**. This large difference indicates that the model performs very well on the training data but poorly on unseen test data. This situation usually suggests that the model is **overfitting** and is unable to generalize effectively.

Understanding MSE

Mean Squared Error (MSE) is a performance metric used to measure the average squared difference between the predicted values and the actual values.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- **Lower MSE** indicates better prediction accuracy.
- **Training MSE = 0.85** means the model fits the training data very well.
- **Testing MSE = 5.20** means prediction errors are much larger on new data.

The large gap between these values shows that the model has learned the training data too specifically instead of learning general patterns.

Possible Reasons for the Difference

1. Overfitting
Overfitting is the most common reason for a low training error and a high testing error. The neural network memorizes the training examples instead of learning the underlying relationship between inputs and outputs.
Example:
If the model is trained only on one group's exam patterns, it may fail to predict scores accurately for students from another group.

2. Small Training Dataset
If the training dataset contains only a limited number of student records, the neural network does not get enough examples to learn all possible patterns. As a result, it performs poorly on unseen data.

3. Complex Neural Network
Using too many hidden layers or too many neurons increases the complexity of the model. A highly complex model can easily memorize the training data, leading to poor generalization.

4. Poor Quality Data
The dataset may contain:

- Missing values
- Incorrect student scores
- Duplicate records
- Outliers

These issues reduce the model's ability to learn meaningful patterns.

5. Different Training and Testing Data
If the training and testing datasets are not from the same distribution, the model cannot predict accurately.
Example:

- Training data contains students from one school.
- Testing data contains students from another school with different grading methods.

6. Noise in the Dataset
Some student scores may be affected by random factors or incorrect entries. The model may learn these noisy patterns instead of useful information.

7. Improper Feature Selection
Important features such as:

- Attendance
- Previous semester marks
- Assignment scores
- Study hours

may be missing from the dataset. Without relevant features, prediction accuracy decreases.

8. Lack of Data Normalization
Neural networks perform better when input values are normalized or standardized. If one feature has much larger values than another, training becomes less effective.

Corrective Measures

1. Use Regularization

Regularization techniques reduce overfitting.

Examples:

- L1 Regularization
- L2 Regularization (Weight Decay)

These methods prevent the model from learning unnecessary details.

2. Apply Dropout

Dropout randomly disables some neurons during training.

Benefits:

- Prevents memorization
- Improves generalization
- Reduces overfitting

3. Increase Training Data

Collect more student records from different schools or academic years.

Advantages:

- Better learning
- Improved prediction accuracy
- Reduced overfitting

4. Early Stopping

Stop training when the validation error starts increasing instead of continuing until the training error becomes very low.

This prevents the model from over-learning the training data.

5. Simplify the Neural Network

Reduce:

- Number of hidden layers
- Number of neurons
- Total parameters

A simpler model often performs better on unseen data.

6. Improve Data Quality

Before training:

- Remove duplicate records
- Handle missing values
- Correct incorrect entries
- Remove outliers

Clean data leads to better predictions.

7. Normalize the Data

Scale all input features using:

- Min-Max Normalization
- Standardization (Z-score)

This helps the neural network learn more efficiently.

8. Use Cross-Validation

K-Fold Cross Validation evaluates the model on different subsets of the dataset.

Benefits:

- More reliable performance estimate
- Better model selection
- Reduced bias

9. Feature Engineering

Include more useful input features such as:

- Attendance percentage
- Internal assessment marks
- Study hours
- Assignment completion
- Previous academic performance

These features improve prediction accuracy.

10. Hyperparameter Tuning

Adjust important parameters such as:

- Learning rate
- Batch size
- Number of epochs
- Number of hidden neurons
- Activation functions

Proper tuning improves model performance on both training and testing data.

Conclusion

The neural network achieves a **low training MSE (0.85)** but a **much higher testing MSE (5.20)**, indicating that it does not generalize well to unseen data. The most likely cause is **overfitting**, although factors such as insufficient training data, poor data quality, complex model architecture, lack of normalization, and improper feature selection may also contribute. These issues can be addressed by using **regularization, dropout, early stopping, cross-validation, improved data preprocessing, feature engineering, hyperparameter tuning, and collecting more representative training data**. Applying these corrective measures will help reduce the testing MSE and improve the overall accuracy and reliability of the neural network for predicting student scores.

Code of Conduct:

*I **K Gowthami (192525428)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K Gowthami

RUBRICS FOR CALCULATION

Criteria	**Weightage**	**Level 4 – Exemplary (5)**	**Level 3 – Proficient (4)**	**Level 2 – Developing (2-3)**	**Level 1 – Beginning (0-1)**
Error Identification	30%	Correctly identifies all errors and their causes	Identifies most errors with minor omissions	Identifies some errors but misses key issues	Unable to identify errors
Error Analysis & Reasoning	30%	Provides clear and logical explanation of why errors occurred	Reasoning mostly corrects with minor gaps	Limited reasoning and partial explanation	No meaningful analysis
Correction Strategy	25%	Suggests appropriate corrections with justification	Suggests correct corrections with minor gaps	Partially correct corrections	Incorrect or no corrections
Interpretation of Results	15%	Clearly explains impact of errors on model performance/results	Basic interpretation with minor omissions	Limited interpretation	No interpretation provided